

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Разработка мобильного приложения-библиотеки «Liboffan»

Курсовая работа

09.03.02 Информационные системы и технологии

Профиль «Программная инженерия в информационных системах»

Зав. кафедрой

Зав. кафедрой _____ Махортов С. Д. д. ф-м.н., профессор

Обучающийся _____ Щеблыкина А. В.

Руководитель _____ Тарасов В. С.

Консультант _____ Ушаков В. А.

_____ Махортов С. Д. д. ф-м.н., профессор __.__.20__

Обучающийся _____ Щеблыкина А. В.

Руководитель _____ Тарасов В. С. ст. преподаватель

Воронеж 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
РЕФЕРАТ	4
ВВЕДЕНИЕ.....	5
1 Предметная область и существующие решения	7
1.1 Описание предметной области и актуальности	7
1.2 Общее описание рынка платформ для чтения и творчества.....	8
1.3 Обзор аналогов	9
1.3.1 Wattpad.....	9
1.3.2 Author.Today.....	10
1.3.3 Ficbook.....	11
1.3.4 Результат сравнения аналогов.....	12
2 Проектирование системы	15
2.1 Пользовательские сценарии	15
2.2 Средства реализации.....	18
2.3 Архитектура разрабатываемой системы.....	20

2.3.1 Серверная часть приложения.....	20
2.3.2 База данных	23
2.3.3 Клиентская часть приложения.....	24
2.3.4 Взаимодействие клиент-сервер	28
3 Реализация разрабатываемой системы	30
3.1 Серверная часть приложения.....	30
3.2 Клиентская часть приложения.....	33
3.2.1 Экран авторизации.....	33
3.2.2 Экран создания истории	34
3.2.3 Главный экран (Каталог историй)	36
3.2.4 Экран поиска	37
3.2.5 Экран профиля и библиотеки	39
3.2.6 Экран AI-редактирования	40
ЗАКЛЮЧЕНИЕ	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	44

РЕФЕРАТ

Бакалаврская работа 46 с., 18 рис., 1 табл., 11 использованных источников.

МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, КОЛЛАБОРАТИВНОЕ ТВОРЧЕСТВО, ВЕТВЛЕНИЕ СЮЖЕТОВ, ВЕРСИОНИРОВАНИЕ КОНТЕНТА, AI-РЕДАКТИРОВАНИЕ, ANDROID, KOTLIN, JETPACK COMPOSE

Объект исследования – мобильное приложение для коллаборативного чтения и создания историй с поддержкой нелинейного повествования и системой управления версиями текста.

Цель исследования – разработка платформы, позволяющей пользователям не только потреблять литературный контент, но и активно участвовать в его развитии через создание альтернативных версий, ответвлений сюжетов и совместное редактирование с использованием искусственного интеллекта.

Результаты работы: исследован рынок цифровых библиотек и платформ пользовательского творчества. Разработано мобильное Android-приложение «Liboffan», реализующее механику ветвления историй, систему авторства версий, персональную библиотеку с отслеживанием статуса чтения, а также интеграцию локального AI-ассистента для генерации и стилистического редактирования текста. Приложение обеспечивает удобную навигацию по дереву сюжетов без потери связи с оригинальным произведением.

Область применения результатов: сервисы цифровой публикации, сообщества авторов и читателей (включая фанфикшн и оригинальную прозу), образовательные платформы для развития литературных навыков, а также конечные пользователи, заинтересованные в интерактивном, нелинейном формате чтения и совместного творчества.

ВВЕДЕНИЕ

В условиях цифровизации современного общества и повсеместного распространения мобильных устройств платформы для чтения и творчества претерпевают значительные изменения. Традиционные модели литературного творчества, предполагающие единственного автора и линейное повествование, уступают место коллаборативным формам, где множество авторов могут развивать одну и ту же идею в разных направлениях. Современные читатели всё чаще стремятся не просто потреблять контент, но и участвовать в его создании, предлагать альтернативные варианты развития сюжета, экспериментировать с жанрами и стилями. Однако существующие платформы для чтения и публикации текстов либо предлагают пассивное потребление контента, либо не предоставляют удобных инструментов для совместного творчества и ветвления историй.

Особую актуальность приобретает концепция «версионирования» литературных произведений — возможность создавать альтернативные версии текста, сохраняя связь с оригиналом и позволяя читателям выбирать предпочтительное развитие событий. Такой подход, заимствованный из практики разработки программного обеспечения (системы контроля версий Git), открывает новые возможности для литературного творчества и чтения. Таким образом, разработка мобильного приложения для коллаборативного создания историй с системой ветвления сюжетов является актуальной задачей, отвечающей современным тенденциям развития цифрового творчества и чтения. Целью данной курсовой работы является создание мобильного Android-приложения «Liboffan», предоставляющего платформу для совместного создания и чтения историй с возможностью ветвления сюжетов и использования AI-помощника для редактирования текстов. Задачами работы являются:

Анализ предметной области и существующих решений:

- исследование рынка платформ для чтения и пользовательского творчества;
- выявление функциональных возможностей и ограничений аналогов; определение требований к разрабатываемой системе;
- выбор средств разработки и технологий;
- обоснование выбора Kotlin и Jetpack Compose для клиентской части; выбор Spring Boot для серверной части;

Проектирование структуры базы данных для хранения версий историй; проектирование архитектуры приложения:

- разработка клиент-серверной архитектуры;
- проектирование базы данных с поддержкой версионирования; определение пользовательских сценариев и навигации;
- реализация функций разрабатываемой системы;
- реализация серверной части с REST API;
- создание клиентской части с современным пользовательским интерфейсом;
- интеграция AI-помощника на основе Ollama для редактирования текстов;
- тестирование основных пользовательских сценариев.

Практическая значимость работы заключается в создании полнофункциональной платформы, которая закрывает существующий пробел между пассивным чтением и активным творчеством, предоставляя пользователям удобные инструменты для создания альтернативных версий любимых историй и коллаборативного развития сюжетов.

1 Предметная область и существующие решения

1.1 Описание предметной области и актуальности

Предметной областью разрабатываемой системы является мобильное приложение для коллаборативного создания и чтения историй с системой ветвления сюжетов. В отличие от традиционных платформ для публикации текстов, где каждый автор создаёт изолированное произведение, Liboffan позволяет строить разветвлённую структуру историй, где каждое ответвление сохраняет связь с родительской версией.

Ключевой особенностью системы является концепция «дерева историй» (story tree) — структуры, в которой корневая версия представляет оригинальное произведение, а дочерние версии (ветки) являются альтернативными продолжениями или интерпретациями, созданными другими авторами. При этом каждая версия сохраняет авторство, имеет собственное описание, теги и возрастной рейтинг.

Система поддерживает гибридные описания: у дерева историй есть общий синопсис, но каждая конкретная ветка может иметь своё уникальное описание, отражающее взгляд её автора на развитие сюжета. Это позволяет читателям осознанно выбирать между различными вариантами повествования.

Для навигации и поиска используется система тегов и фандомов, а также возрастные рейтинги (G, PG, PG-13, R, NC-17), что обеспечивает удобную фильтрацию контента по интересам и предпочтениям пользователей.

Разрабатываемое приложение можно классифицировать как платформу коллаборативного творчества с элементами социального чтения, объединяющую функции:

- чтения и публикации текстов;
- версионирования и ветвления историй;
- управления личной библиотекой;
- социального взаимодействия (лайки, подписки);

- AI-ассистента для редактирования текстов.

Таким образом, Liboffan представляет собой инновационное решение, объединяющее лучшие практики из мира разработки программного обеспечения (системы контроля версий) и литературного творчества, создавая новую модель взаимодействия авторов и читателей.

1.2 Общее описание рынка платформ для чтения и творчества

Рынок цифровых платформ для чтения и литературного творчества демонстрирует устойчивый рост, однако большинство существующих решений ориентировано либо на пассивное потребление контента, либо на публикацию изолированных произведений. Ключевые тенденции включают развитие социальных функций (комментарии, рейтинги, подписки), внедрение инструментов для авторов (статистика, монетизация) и персонализацию рекомендаций на основе предпочтений пользователей.

Вместе с тем, наблюдается дефицит платформ, поддерживающих коллаборативное творчество и нелинейное развитие сюжетов. Большинство сервисов не предоставляют механизмов для создания альтернативных версий текста с сохранением связи с оригиналом, что ограничивает возможности экспериментов и совместной работы авторов.

Особую нишу занимают платформы для фанатского творчества, однако они часто фрагментированы, имеют устаревший интерфейс и не предлагают удобных инструментов для навигации по разветвлённым сюжетам. В связи с этим возрастает потребность в платформах, обеспечивающих прозрачное отслеживание авторства и техническую взаимосвязь между оригинальным текстом и пользовательскими адаптациями. Разрабатываемое приложение «Liboffan» позиционируется как решение, объединяющее преимущества традиционных читалок, социальных сетей и систем контроля версий, создавая принципиально новый формат взаимодействия с литературным контентом.

1.3 Обзор аналогов

Для выявления функциональных требований и конкурентных преимуществ были проанализированы три ключевых платформы: Wattpad, Author.Today и Ficbook. Критерии сравнения включали:

- поддержку ветвления сюжетов;
- инструменты совместного творчества;
- систему тегов и фандомов;
- наличие мобильных приложений и возможности персонализации.

1.3.1 Wattpad

Wattpad — крупнейшая международная платформа для публикации пользовательских историй. Приложение предлагает обширную библиотеку контента, социальные функции (комментарии, списки чтения) и инструменты для авторов. Однако ветвление сюжетов реализовано лишь в виде сиквелов и приквелов — отдельных произведений, не связанных технической зависимостью. Альтернативные версии одной истории приходится искать вручную через теги, что неудобно для читателя. Иллюстрация главного экрана приложения представлена на рисунке 1.

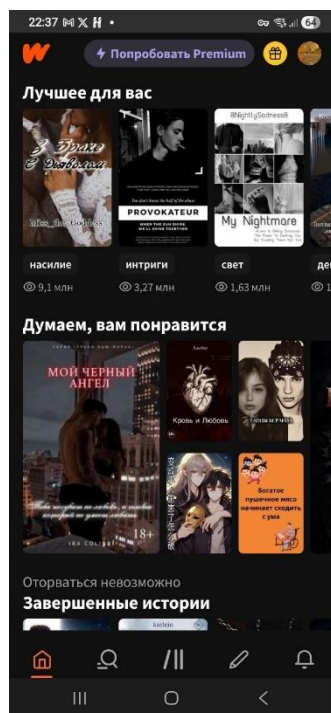


Рисунок 1 - Главная страница приложения Wattpad

1.3.2 Author.Today

Author.Today — российская платформа, ориентированная на профессиональных авторов и платный контент. Сервис предоставляет удобные инструменты публикации, черновики, статистику чтения. Вместе с тем, функционал полностью сосредоточен на линейном повествовании: каждый текст существует в единственном варианте, возможность создания ответвлений или альтернативных концовок отсутствует. Иллюстрация главного экрана приложения представлена на рисунке 2.

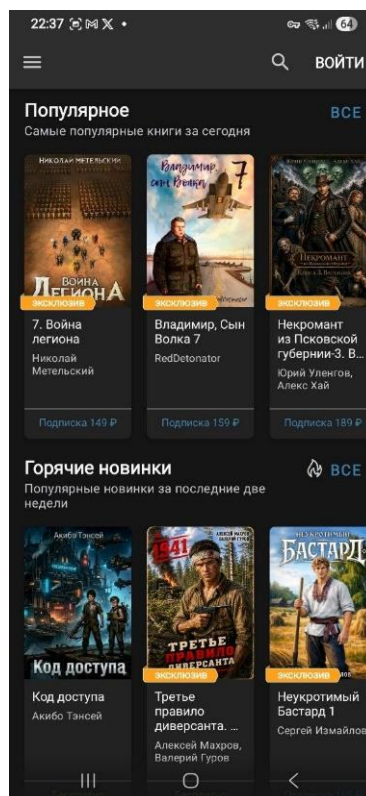


Рисунок 2 - Главная страница приложения Author.Today

1.3.3 Ficbook

Ficbook (Книга Фанфиков) — специализированная платформа для фанатского творчества. Поддерживает развитую систему тегов, предупреждений о контенте, рейтингов. Пользователи могут публиковать альтернативные версии канона, но технически каждая такая работа является отдельным произведением без автоматической связи с оригиналом. Навигация по «деревьям» фанфиков осуществляется вручную через ссылки в описании. Иллюстрация главного экрана приложения представлена на рисунке 3.



Рисунок 3 - Главный экран приложения Ficbook

1.3.4 Результат сравнения аналогов

Результаты сравнения функциональных возможностей аналогов представлены в таблице 1.

Приложение	Ветвление сюжетов	Совместное творчество	Теги и фан-домы	AI-инструменты	Персональная библиотека
Wattpad	– (только сиквелы)	–	+	–	+
Author.Today	–	–	+	–	+
Ficbook	– (ручные ссылки)	– (соавторство)	+	–	+

Liboffan	+	+	+	+	+
----------	---	---	---	---	---

Таблица 1 – Сравнение функциональных возможностей аналогов

Анализ показал, что ни одна из рассмотренных платформ не предоставляет встроенной системы версионирования историй, где ветки технически связаны с родительской версией, а навигация между ними интуитивно понятна. Wattpad и Author.Today ориентированы на линейное повествование, где каждое произведение существует изолированно. Ficbook, несмотря на развитую систему тегов и поддержку фанатского творчества, не обеспечивает автоматическую связь между альтернативными версиями — навигация осуществляется вручную через ссылки в описании.

Кроме того, ни один из аналогов не интегрирует AI-помощника для редактирования текстов непосредственно в интерфейс создания контента, что ограничивает возможности авторов по быстрой доработке и улучшению своих произведений.

Выявленные функциональные пробелы определяют конкурентные преимущества разрабатываемой системы «Liboffan».

По результатам анализа аналогов был сформирован следующий перечень ключевых функций приложения:

- создание оригинальных историй с указанием названия, синопсиса, контента и метаданных (возрастной рейтинг, теги, фандомы);
- создание ответвлений (веток) от существующих версий с возможностью переопределения описания и тегов;
- автоматическое сохранение иерархической связи между версиями через поле `parent_id`;
- просмотр дерева историй с навигацией между корневой версией и ответвлениями;

- гибридные описания: общее описание дерева + уникальное описание каждой ветки;
- чтение версий с настраиваемым размером;
- сохранение конкретной версии в библиотеке (не дерева);
- постановка лайков на конкретные версии;
- подписка на авторов и отслеживание новых работ;
- поиск по названию, тегам, фандомам, возрастному рейтингу;
- фильтрация результатов поиска по нескольким критериям одновременно;
- выделение фрагмента текста и отправка в AI-помощник для улучшения стиля или продолжения текста;
- генерация продолжения текста AI-помощником на основе существующего контекста;
- предпросмотр предложений AI перед применением изменений;

2 Проектирование системы

2.1 Пользовательские сценарии

Проектирование системы начиналось с определения основных ролей и сценариев использования. В системе выделены две ключевые роли: Читатель и Автор, причём любой зарегистрированный пользователь может совмещать обе функции.

Базовые сценарии для Читателя включают:

- регистрацию и авторизацию,
- просмотр каталога историй, поиск по тегам и фанdomам,
- чтение версий (как корневых, так и ответвлений), управление личной библиотекой (добавление со статусами «Планирую», «Читаю», «Завершено», «Брошено»),
- постановку лайков и подписку на авторов.

Сценарии для Автора расширяют функционал:

- создание оригинальной истории с указанием названия, синопсиса, контента и метаданных;
- создание ответвления от существующей версии с возможностью переопределения описания и тегов;
- редактирование собственных версий;
- просмотр статистики по своим произведениям.

Отдельный сценарий — использование AI-помощника:

- выделение фрагмента текста, отправка запроса на улучшение стиля,
- продолжение сюжета или сокращение,
- предпросмотр предложения и применение изменений.

Этот сценарий доступен как при создании оригинала, так и при написании ответвления.

Все сценарии объединены в единый пользовательский путь. Навигация между экранами спроектирована таким образом, чтобы минимизировать количество переходов для выполнения ключевых действий.

Были сформированы пользовательские сценарии с помощью диаграммы прецедентов. Спроектированная диаграмма прецедентов представлена на рисунке 4.

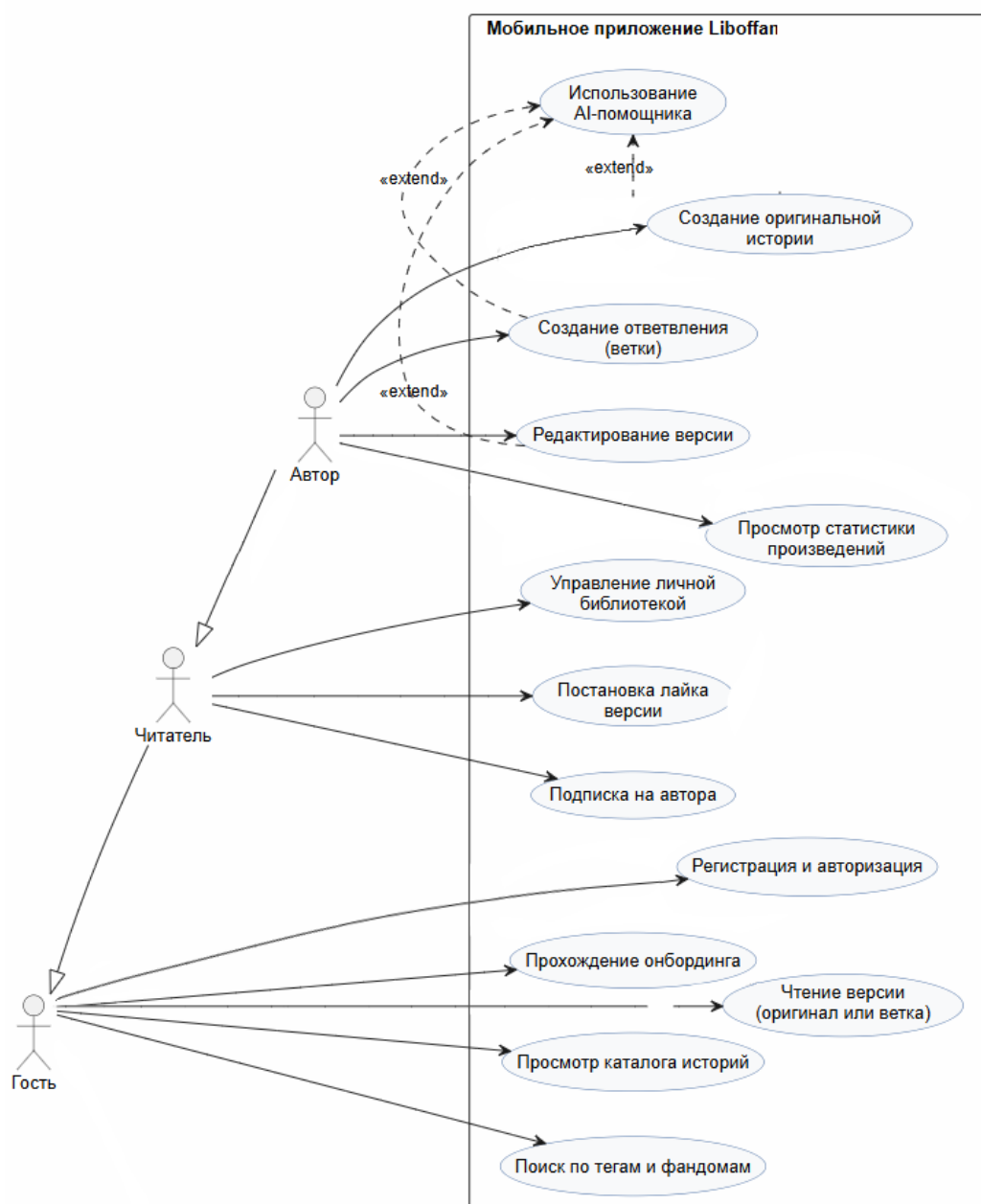


Рисунок 4 - Диаграмма прецедентов

В системе предусмотрены три типа пользователей: Гость (неавторизованный пользователь), Читатель и Автор (авторизованные пользователи). При этом любой авторизованный пользователь может совмещать роли Читателя и Автора.

Неавторизованный пользователь (Гость) имеет ограниченный доступ: просмотр каталога историй, поиск по тегам и фанdomам, чтение опубликованных версий. Для доступа к полному функционалу необходимо пройти регистрацию с указанием email и пароля. При последующих запусках приложения осуществляется автоматическая авторизация по сохранённому JWT-токену.

Основные пользовательские сценарии приложения:

- Прохождение онбординга — знакомство с ключевыми возможностями приложения при первом запуске;
- Регистрация и авторизация — создание аккаунта или вход с использованием email и пароля;
- Просмотр каталога историй — отображение последних добавленных произведений на главном экране;
- Поиск по тегам, фанdomам и возрастным рейтингам — фильтрация историй по нескольким критериям одновременно;
- Чтение версии — просмотр текста как корневой версии (оригинала), так и ответвлений с переключением между ними;
- Создание оригинальной истории — публикация нового произведения с указанием названия, синопсиса, контента, тегов и фанdomов;
- Создание ответвления (ветки) — написание альтернативной версии существующей истории с возможностью переопределения описания;
- Управление личной библиотекой — добавление историй со статусами «Планирую», «Читаю», «Завершено», «Брошено»;
- Постановка лайков — оценка конкретных версий историй;

- Подписка на авторов — отслеживание новых работ любимых писателей;
- Использование AI-помощника — выделение фрагмента текста,правка запроса на улучшение стиля, продолжение или сокращение, предпросмотр и применение изменений;
- Редактирование профиля — изменение имени, даты рождения, адреса электронной почты;
- Загрузка и удаление аватара — персонализация профиля;
- Просмотр статистики — отображение количества прочитанных историй, созданных версий, лайков;
- Выход из аккаунта — завершение сессии с очисткой токена.

2.2 Средства реализации

Для реализации клиентской (Android) части приложения выбраны инструменты и технологии, представленные ниже.

Язык программирования Kotlin — современный статически типизированный язык программирования для JVM-платформы, официально рекомендуемый Google для разработки Android-приложений. Kotlin обеспечивает лаконичность кода, безопасность в отношении нулевых ссылок и полную совместимость с Java;[1]

Jetpack Compose — современный инструмент для создания пользовательских интерфейсов на Android, который использует декларативный подход. Разработчик описывает, как должен выглядеть UI в зависимости от данных, а система автоматически обновляет его при изменении этих данных. Позволяет описывать пользовательский интерфейс в виде функций-компонентов на Kotlin, использует Material Design 3 в качестве дизайн-системы, обеспечивает реактивное обновление интерфейса при изменении состояния; [2]

Паттерн MVVM (Model-View-ViewModel) позволяет разделить логику приложения от визуальной части (представления). Данный паттерн является архитектурным, то есть он задает общую архитектуру приложения. ViewModel хранит состояние экрана и предоставляет UI-компонентам необходимые данные через StateFlow. Это повышает тестируемость и масштабируемость кода; [3]

Retrofit — это библиотека, которая упрощает работу с сетевыми запросами в приложениях на Android. Позволяет описывать REST API в виде интерфейсов, автоматически преобразует JSON-ответы в Kotlin-объекты и интегрируется с окклюзией (OkHttp) для управления сетевыми запросами; [4]

Для реализации серверной части приложения выбраны средства, описанные ниже.

Язык программирования Java — используется для написания серверной логики, обеспечивая строгую типизацию и широкую поддержку в корпоративной среде;

Фреймворк Spring Boot 3 — платформа для создания автономных, готовых к работе приложений на Java. Включает встроенный сервер Tomcat, упрощает конфигурацию зависимостей и предоставляет готовые решения для безопасности и работы с данными; [5]

Spring Security + JWT — модуль безопасности, обеспечивающий аутентификацию и авторизацию пользователей посредством JSON Web Tokens (JWT), что позволяет реализовывать stateless-сессии; [6]

СУБД MariaDB — реляционная система управления базами данных с широкой распространённостью и развитой экосистемой. Обеспечивает надёжное хранение структурированных данных о пользователях, деревьях историй, версиях, лайках и библиотеках; [7]

Spring Data JPA — абстракция для работы с базами данных в экосистеме Spring. Позволяет создавать репозитории на основе интерфейсов, автоматически генерируя SQL-запросы для CRUD-операций; [8]

Ollama — инструмент для запуска больших языковых моделей (LLM) локально. Используется для интеграции AI-помощника, позволяющего генерировать и редактировать текст непосредственно в приложении без отправки данных в облако; [9]

2.3 Архитектура разрабатываемой системы

Мобильное приложение «Liboffan» построено на архитектуре клиент-серверного взаимодействия. Android-приложение выступает в роли клиента и взаимодействует с серверной частью через REST API по протоколу HTTPS.

2.3.1 Серверная часть приложения

Серверная часть приложения реализована на фреймворке Spring Boot и организована по многоуровневой архитектуре (Controller – Service – Repository). Сервер предоставляет набор REST API эндпоинтов, сгруппированных по функциональным модулям. Структура серверной части представлена на рисунке 5.

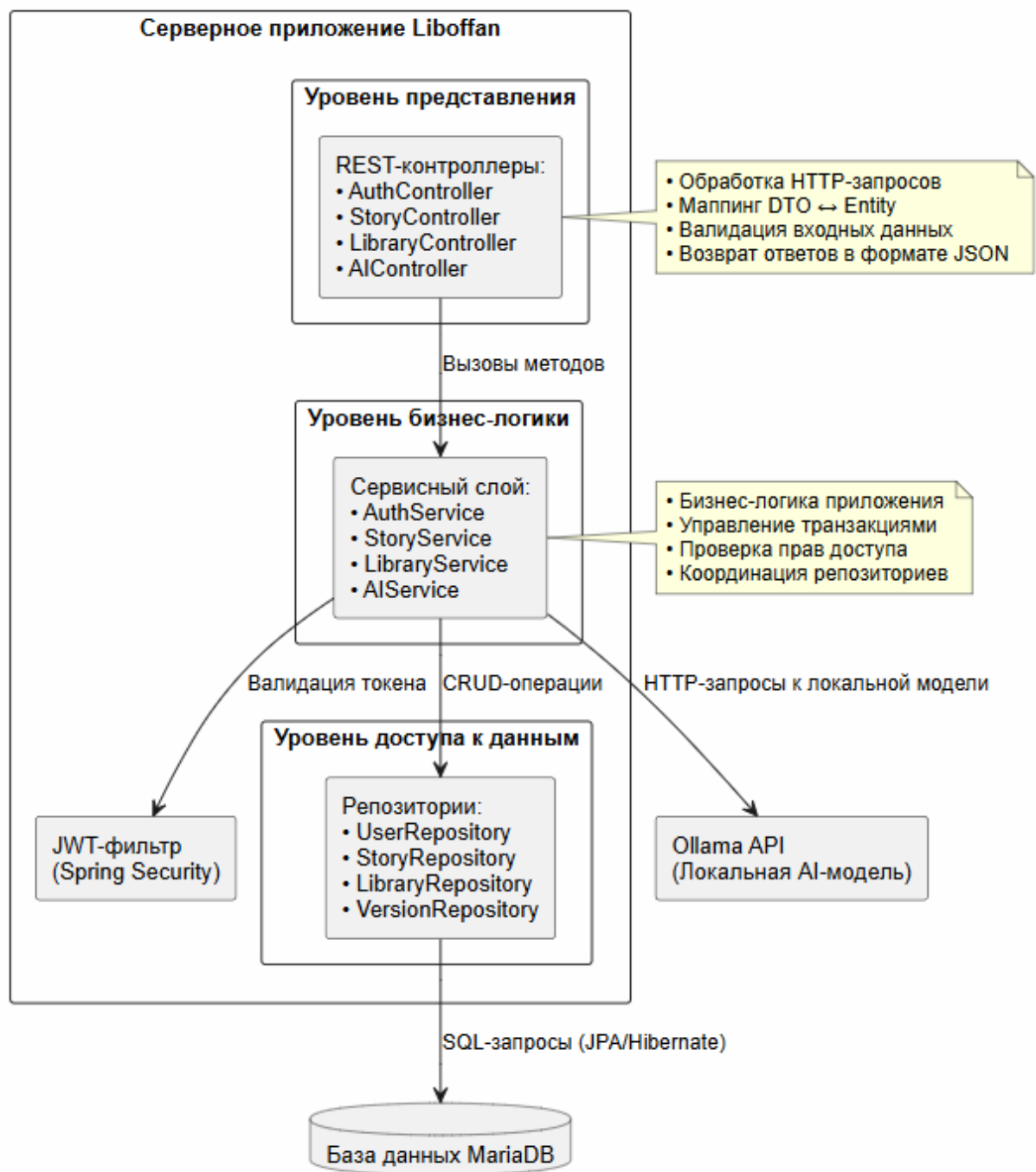


Рисунок 5 - Многоуровневая архитектура серверной части

Серверное приложение включает следующие основные модули:

- AuthModule — маршруты для регистрации и авторизации пользователей (POST /api/auth/login, POST /api/auth/register);

- StoryModule — маршруты для работы с деревьями историй и их версиями (GET /api/stories, POST /api/stories, POST /api/stories/{treeId}/fork, GET /api/stories/versions/{versionId});
- LibraryModule — маршруты для управления личной библиотекой пользователя (POST /api/me/library, DELETE /api/me/library/{versionId}, GET /api/me/full);
- ReferenceModule — справочные маршруты для получения списков тегов и фандомов (GET /api/tags, GET /api/fandoms);
- AIModule — маршруты для взаимодействия с локальной моделью Ollama (POST /api/ai/edit, POST /api/ai/generate-idea).

Безопасность API обеспечивается посредством JWT-аутентификации. При входе пользователю выдаётся JWT-токен, который клиент прикладывает к каждому последующему запросу в заголовке Authorization. Сервер проверяет валидность токена через фильтр Spring Security при каждом обращении к защищённым эндпоинтам. Пароли пользователей хранятся в базе данных в хэшированном виде с применением алгоритма BCrypt. Схема эндпоинтов API представлена на рисунке 6.

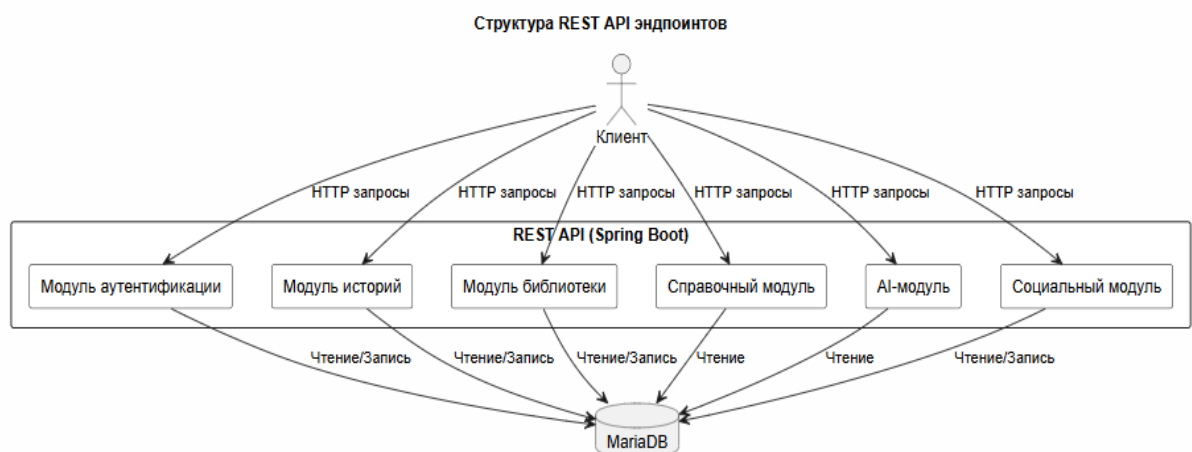


Рисунок 6 - Схема REST API эндпоинтов приложения

2.3.2 База данных

Для хранения данных используется реляционная СУБД MariaDB. База данных спроектирована с приведением к третьей нормальной форме для минимизации избыточности. ER-диаграмма базы данных представлена на рисунке 7.

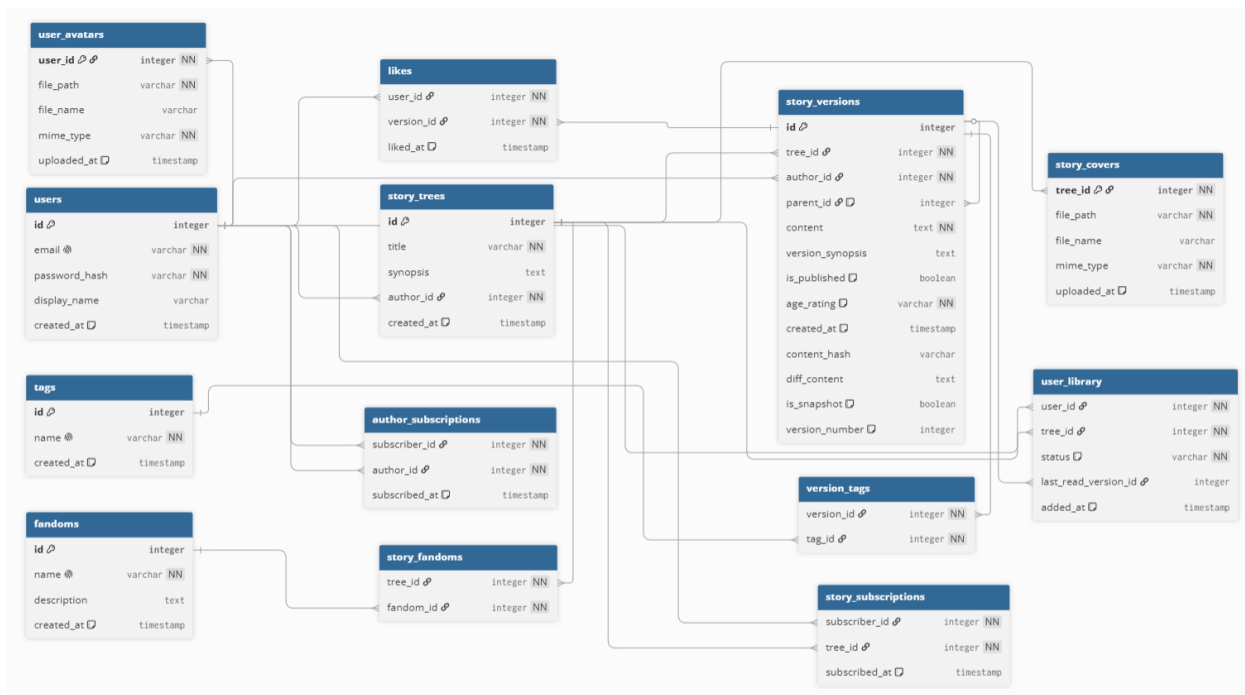


Рисунок 7 - ER-диаграмма базы данных

База данных содержит следующие основные сущности:

- User — данные пользователя (идентификатор, email, хэш пароля, имя, путь к аватару);
- StoryTree — корневая сущность произведения (идентификатор, название, общий синопсис, ID автора, дата создания);
- StoryVersion — конкретная версия текста (идентификатор, ID дерева, ID автора, ID родительской версии parent_id для ветвления, контент, версия синопсиса, рейтинг);

- UserLibrary — записи библиотеки (ID, ID пользователя, ID дерева, ID версии, статус чтения, дата добавления);
- Tag / Fandom — справочники меток и фандомов;
- VersionTag — таблица связей для привязки тегов к конкретным версиям;
- Like — лайки пользователей к версиям.

Ключевой особенностью схемы является самореференциальная связь в таблице StoryVersion (поле parent_id), позволяющая строить иерархию ответвлений произвольной глубины.

2.3.3 Клиентская часть приложения

Построение пользовательского интерфейса и управление состоянием экранов реализовано с применением архитектурного подхода MVVM (Model–View–ViewModel), который обеспечивает модульность, тестируемость и поддерживаемость кодовой базы. Архитектура приложения разделена на два основных слоя: слой представления (UI Layer) и слой данных (Data Layer), взаимодействие между которыми осуществляется через чётко определённые интерфейсы. Взаимосвязь между компонентами архитектуры, включая потоки данных и обработку пользовательских событий, представлена на рисунке 8.[10]

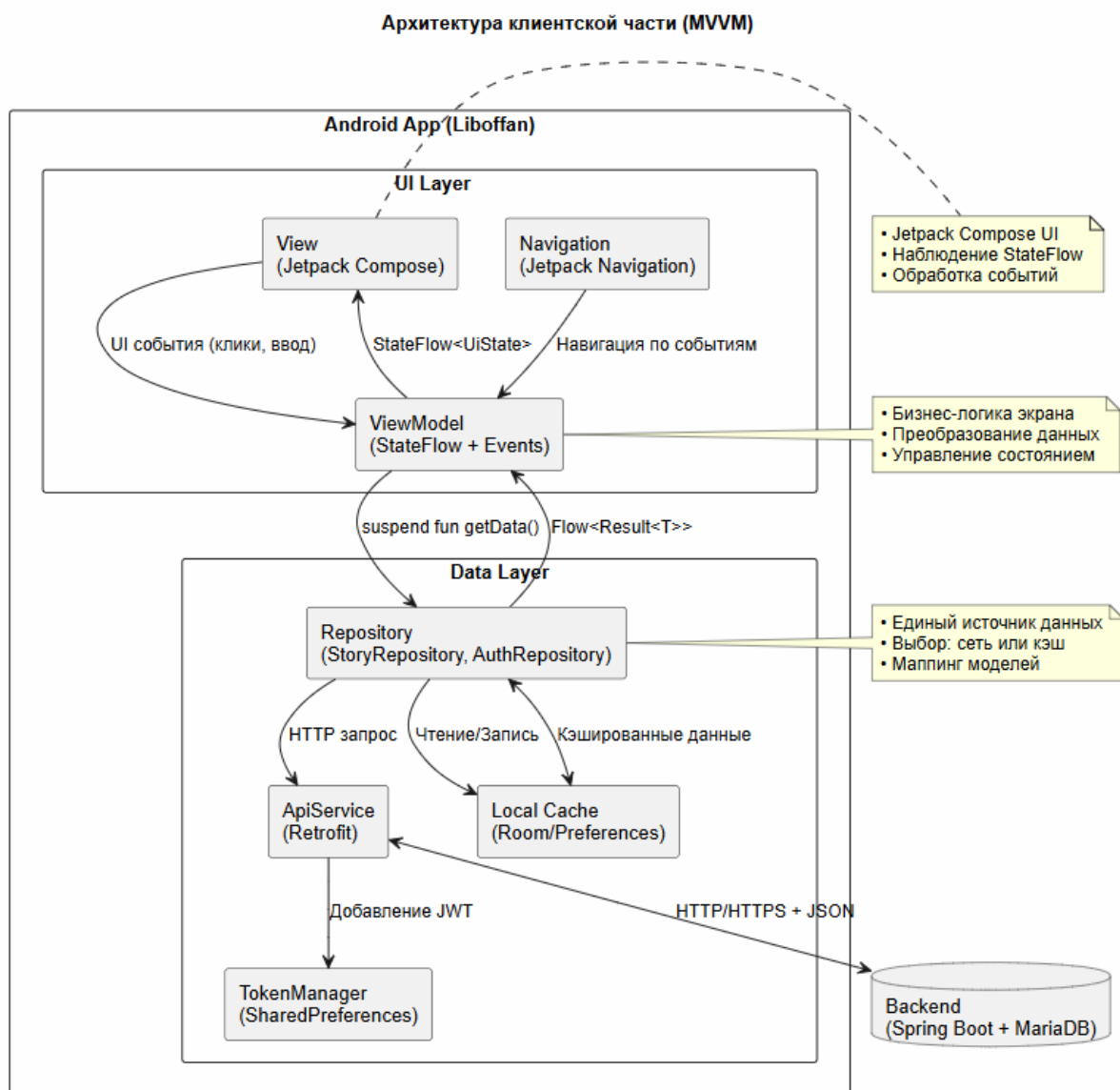


Рисунок 8 - Схема взаимодействия компонентов MVVM-архитектуры

UI Layer (Слой представления) отвечает за отображение пользовательского интерфейса и взаимодействие с пользователем. Данный слой включает три ключевых компонента:

- View (Jetpack Compose) — компоненты пользовательского интерфейса, реализованные в виде Composable-функций. View подписывается на состояние ViewModel через StateFlow и автоматически

ски перерисовывается при изменении данных. Компоненты обрабатывают пользовательские события (клики, ввод текста, жесты) и передают их в ViewModel для обработки;

- ViewModel — содержит бизнес-логику конкретного экрана, управляет состоянием интерфейса (UiState) и координирует получение данных. ViewModel принимает события от View, преобразует их в запросы к слою данных и экспонирует состояние экрана через StateFlow, обеспечивая реактивное обновление UI;
- Navigation (Jetpack Navigation) — компонент навигации, управляющий переходами между экранами приложения. Navigation реагирует на события от ViewModel (например, успешная авторизация или выбор книги) и выполняет переход к соответствующему экрану с передачей необходимых параметров.[11]

Data Layer (Слой данных) предоставляет абстракцию над источниками данных и обеспечивает единый интерфейс для получения и сохранения информации. Слой включает следующие компоненты:

- Repository (StoryRepository, AuthRepository, LibraryRepository) — реализует паттерн Repository, предоставляя единый источник данных для ViewModel. Repository координирует обращение к удалённому API и локальному кэшу, реализуя стратегию «сеть с кэшированием» или «кэш с обновлением». Компонент выполняет маппинг между доменными моделями приложения и моделями данных, возвращая результаты в виде `Flow<Result<T>>`;
- ApiService (Retrofit) — интерфейс для взаимодействия с серверной частью приложения через REST API. Retrofit обеспечивает типобезопасные HTTP-запросы, автоматическую сериализацию/десериализацию JSON и обработку ответов. ApiService определяет все доступные эндпоинты для работы с историями, версиями, библиотекой и аутентификацией;

- TokenManager — компонент для безопасного хранения JWT-токена и данных пользователя на устройстве. TokenManager автоматически добавляет токен авторизации в заголовок каждого защищённого запроса через OkHttp Interceptor, обеспечивая прозрачную аутентификацию;
- Local Cache — локальное хранилище данных для обеспечения работы приложения в офлайн-режиме и ускорения загрузки. Кэш может включать базу данных Room для структурированных данных и DataStore/SharedPreferences для простых настроек и токенов.

Взаимодействие с Backend осуществляется по протоколу HTTP/HTTPS с передачей данных в формате JSON. Серверная часть, реализованная на фреймворке Spring Boot с использованием СУБД MariaDB, предоставляет RESTful API для всех операций приложения.

Маршрутизация внутри приложения реализована с помощью компонента Jetpack Navigation Compose. Вся структура переходов описана в едином навигационном графе AppNavigation, который декларативно задаёт доступные экраны, параметры маршрутов и логику перемещения между ними. Визуальное представление данного графа приведено на рисунке 9.

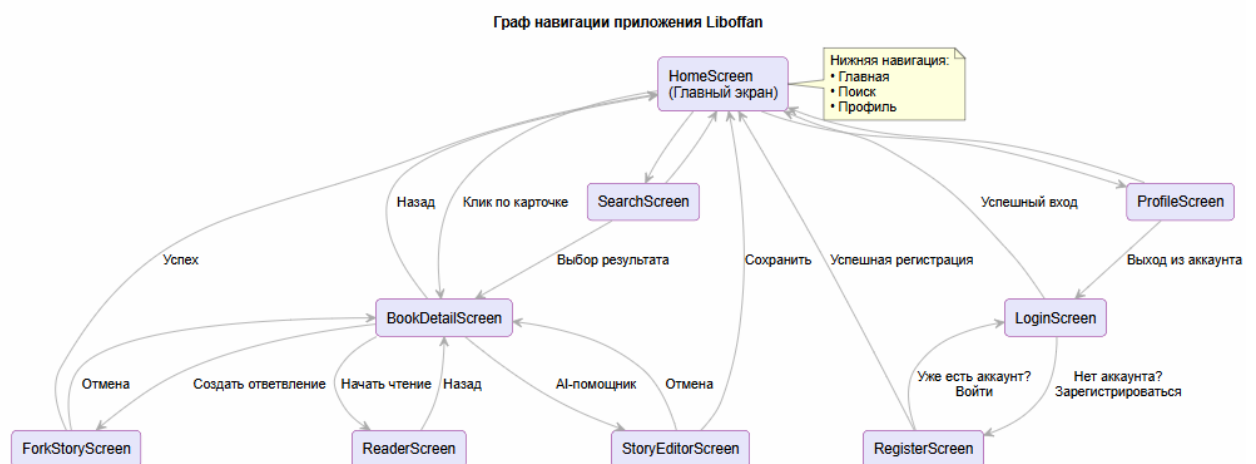


Рисунок 9 - Граф навигации приложения

2.3.4 Взаимодействие клиент-сервер

Обмен данными между клиентской и серверной частями приложения, осуществляемой по протоколу HTTP/HTTPS, где данные передаются в формате JSON, представлены в виде схемы на рисунке 10.

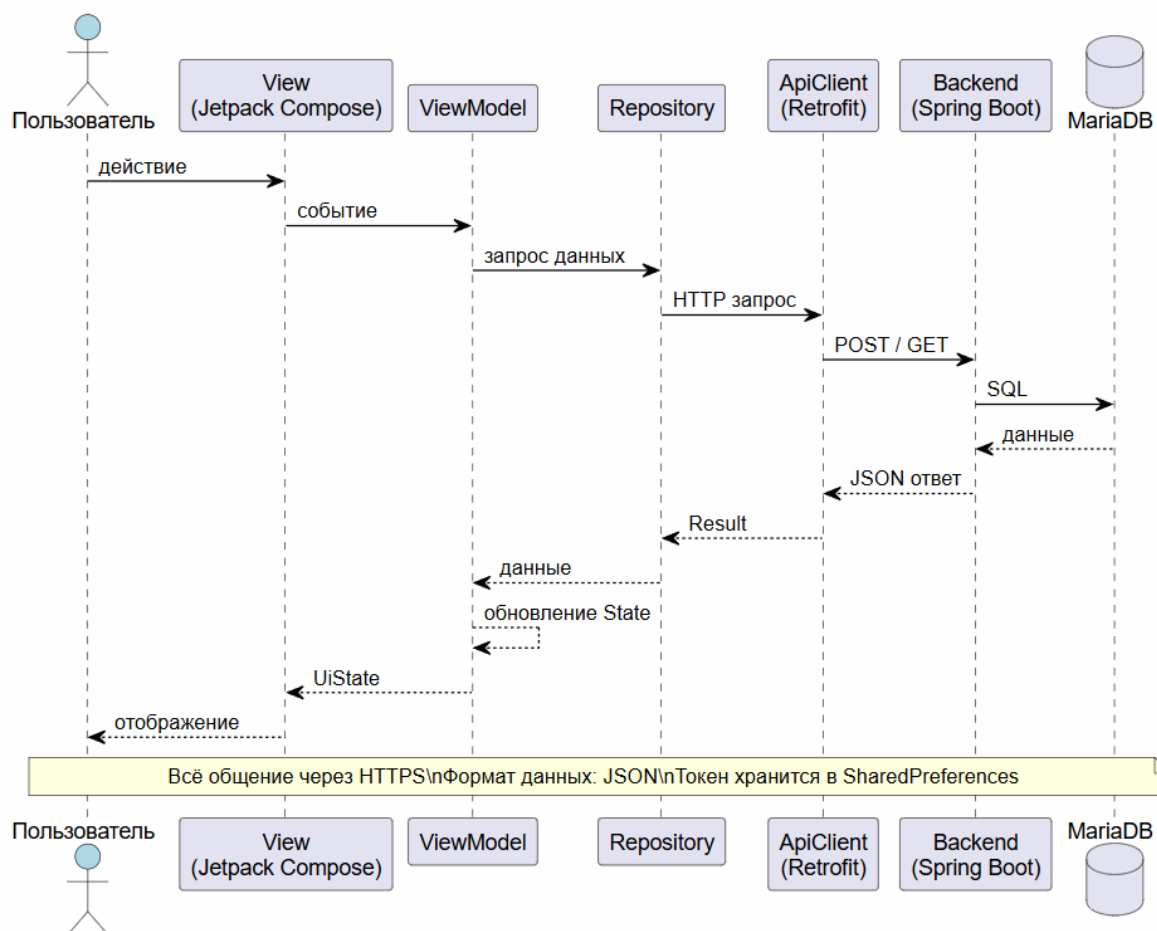


Рисунок 10 - Схема взаимодействия клиент-сервер

Для реализации сетевого уровня на стороне Android применена библиотека Retrofit, которая отвечает за формирование HTTP-запросов и автоматическое преобразование JSON-ответов в типизированные Kotlin-объекты. Механизм аутентификации построен на основе JWT-токенов: при обращении к защищённым ресурсам клиент внедряет токен в заголовок Authorization: Bearer <token>. Управление жизненным циклом токена централизовано в классе

TokenManager, а его автоматическая подстановка в исходящие запросы настроена через перехватчик (Interceptor) библиотеки OkHttp.

На серверной стороне каждый входящий запрос проходит проверку подписи и срока действия токена; при успешной верификации выполняется обработка бизнес-логики и формирование JSON-ответа, в случае ошибки аутентификации клиент получает HTTP-статус 401 Unauthorized.

Для работы AI-функций сервер выступает в роли прокси: клиент отправляет текст и промпт на сервер, сервер делает запрос к локальному экземпляру Ollama (работающему на том же хосте), получает сгенерированный ответ и передаёт его обратно клиенту. Это обеспечивает конфиденциальность данных, так как тексты пользователей не покидают инфраструктуру приложения.

3 Реализация разрабатываемой системы

3.1 Серверная часть приложения

Серверная архитектура приложения «Liboffan» реализована на базе фреймворка Spring Boot с использованием языка Java. Приложение развёрнуто на стандартном порту 8080 и предоставляет Android-клиенту RESTful API, построенное по принципам ресурсно-ориентированного проектирования. Точкой входа выступает класс, аннотированный `@SpringBootApplication`, который инициализирует встроенный servlet-контейнер, подключает модули сериализации (Jackson для JSON), настраивает цепочку фильтров безопасности и регистрирует маршруты контроллеров. Параметры подключения к СУБД MariaDB, стратегии пула соединений и конфигурация JPA вынесены в файл `application.yml`, что обеспечивает гибкость развёртывания в различных окружениях.[11]

Механизм контроля доступа построен на связке Spring Security и JWT-аутентификации. Маршрут регистрации (POST `/api/auth/register`) принимает учётные данные, выполняет проверку уникальности email, применяет алгоритм BCrypt для необратимого хеширования пароля и сохраняет запись пользователя. Эндпоинт входа (POST `/api/auth/login`) верифицирует соответствие пароля сохранённому хешу и генерирует подписанный токен, содержащий идентификатор субъекта и метку времени истечения. Все защищённые ресурсы проходят обработку через кастомный `OncePerRequestFilter`, который извлекает заголовок `Authorization`, проверяет подпись и помещает объект аутентификации в контекст безопасности. Визуализация последовательности операций модуля авторизации приведена на рисунке 11.

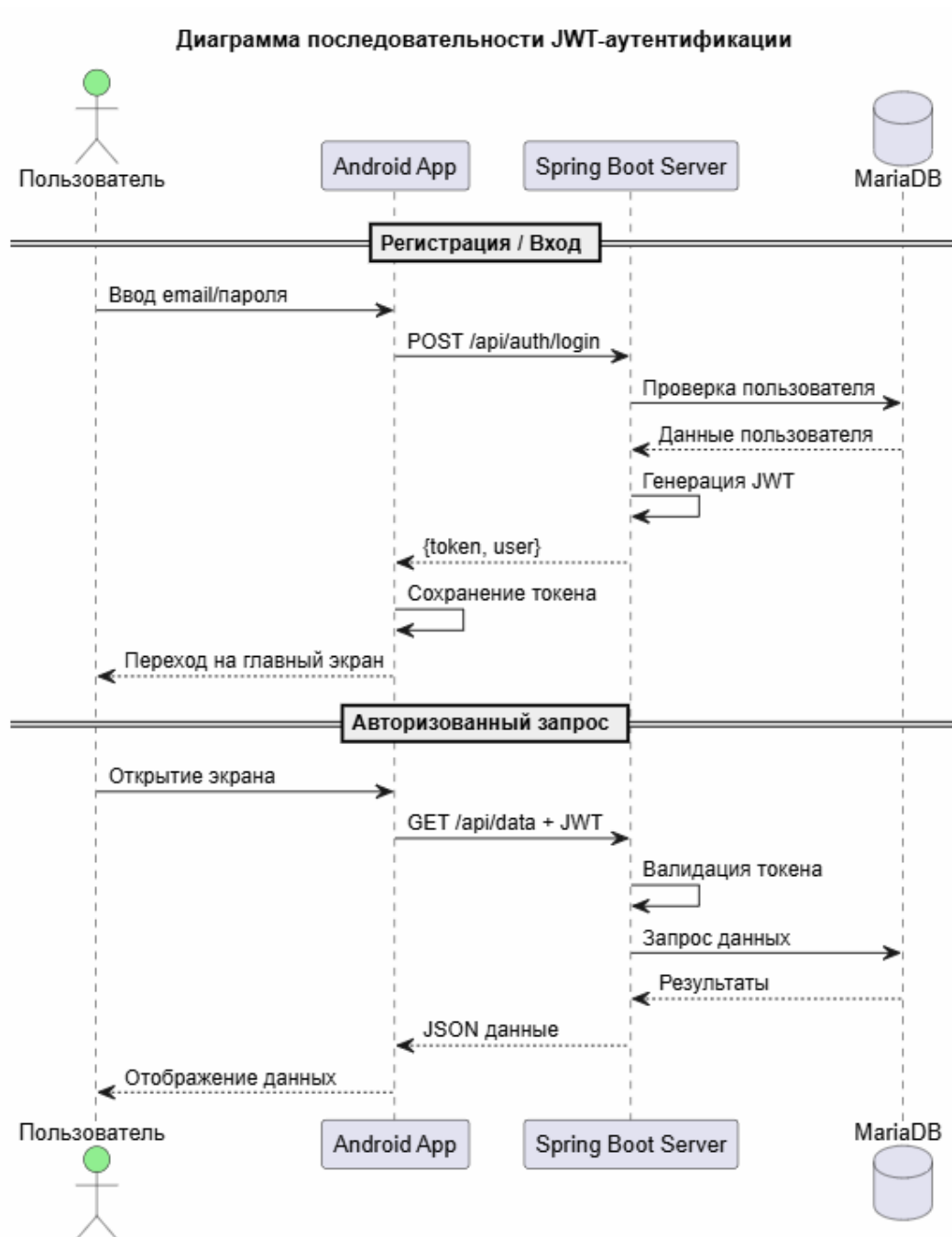


Рисунок 11 - Схема работы модуля авторизации (JWT)

Взаимодействие с искусственным интеллектом реализовано по схеме прокси-сервера. Клиентское приложение не обращается к AI напрямую; вместо этого запрос (выделенный текст и промпт) отправляется на бэкэнд. Сервер, используя HTTP-клиент, пересылает инструкцию локальному экземпляру

Ollama, запущенному в том же контейнере или на соседнем сервере. Визуализация последовательности операций модуля работы с AI приведена на рисунке 12.

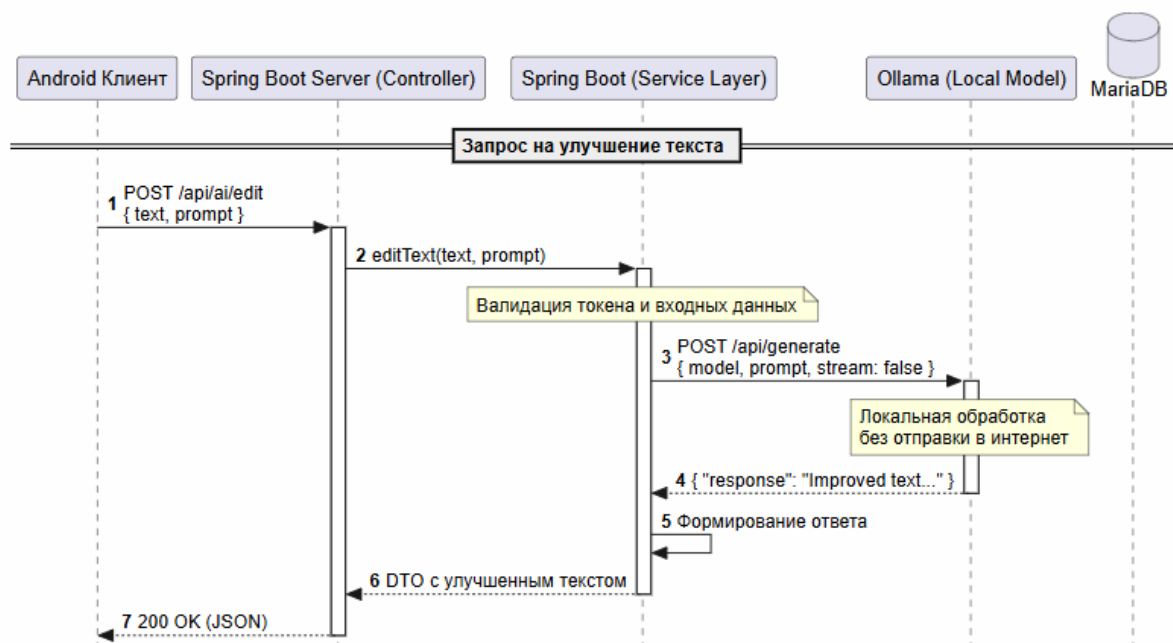


Рисунок 12 - Схема работы модуля работы с AI

Центральный доменный слой отвечает за управление нарративным контентом и поддержку версионности. Создание оригинального произведения (POST /api/stories) инициирует запись в таблицу StoryTree и автоматическую генерацию корневой версии (StoryVersion) с пустым указателем на родителя. Функционал ответвления реализован через маршрут POST /api/stories/{treeId}/fork, который связывает новую запись с существующей через поле parent_id, копирует базовые метаданные и позволяет автору переопределить синопсис, возрастной рейтинг и набор тегов. Чтение контента осуществляется через GET /api/stories/versions/{versionId}, возвращающий детализированный DTO-объект с учётом флага публикации и прав доступа.

Персонализация пользовательского опыта обеспечивается модулем библиотеки и социальными функциями. Добавление произведения в коллекцию

(POST /api/me/library) фиксирует привязку не только к идентификатору дерева, но и к конкретной прочитанной версии, что позволяет точно отслеживать прогресс в разветвлённых сюжетах. Статусы чтения обновляются через PATCH-запросы, а удаление записи выполняется по уникальному идентификатору версии. Социальное взаимодействие включает toggle-механизм лайков (POST /api/stories/versions/{id}/like) и систему подписок на авторов. Управление профилем доступно только аутентифицированным пользователям, чей идентификатор извлекается из JWT-контекста, что исключает возможность несанкционированного изменения чужих данных.

Отдельный интеграционный слой выделен под взаимодействие с локальной языковой моделью. Эндпоинты /api/ai/edit и /api/ai/generate принимают текстовые фрагменты и инструкции, проксируют запросы к HTTP-API экземпляра Ollama, развёрнутого на серверной машине, и возвращают сгенерированные варианты. Подобная архитектура гарантирует, что черновики, личные заметки и незавершённые ветки не покидают периметр инфраструктуры, обеспечивая конфиденциальность творческого процесса и независимость от внешних облачных сервисов.

3.2 Клиентская часть приложения

3.2.1 Экран авторизации

Экран авторизации (LoginScreen) является точкой входа в приложение и предназначен для аутентификации зарегистрированных пользователей. Интерфейс экрана выполнен в минималистичном стиле с использованием градиентного фона фиолетовых оттенков. На экране расположены: заголовок «Добро пожаловать!», текстовые поля для ввода email и пароля (с возможностью переключения видимости пароля), кнопка «Войти», а также ссылки для перехода к экрану регистрации и восстановления пароля. При нажатии кнопки «Войти» выполняется валидация введённых данных: LoginViewModel отправляет

POST-запрос на эндпоинт `/api/auth/login`. В случае успешной аутентификации JWT-токен и данные пользователя сохраняются в `TokenManager`, после чего осуществляется переход на главный экран приложения. Внешний вид экрана авторизации представлен на рисунке 13.

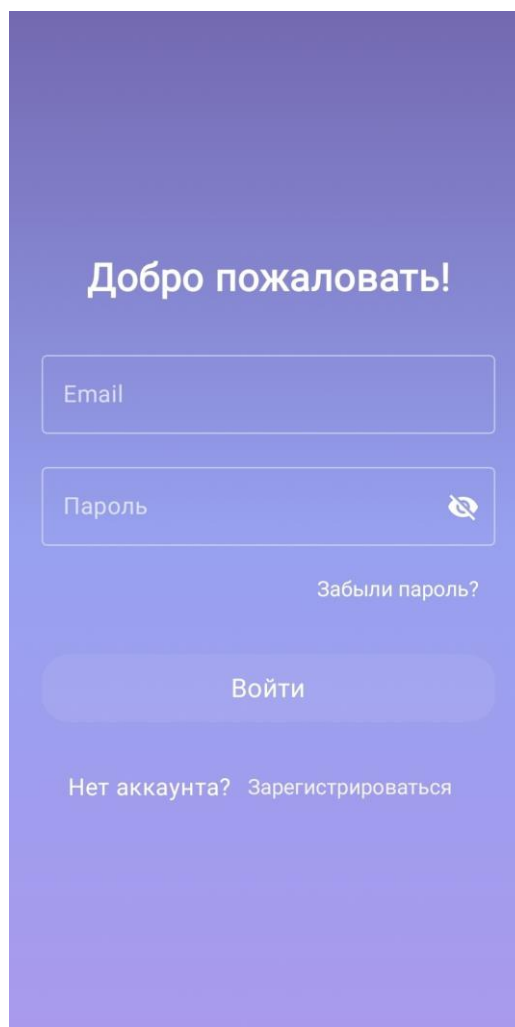


Рисунок 13 - Экран авторизации

3.2.2 Экран создания истории

Экран создания истории (`StoryEditorScreen`) предоставляет авторам инструменты для публикации новых произведений и расширенные инструменты

для создания и улучшения контента с использованием искусственного интеллекта. Интерфейс экрана включает: поле для ввода названия истории (обязательное), текстовую область для краткого описания (синопсиса), выпадающий список для выбора возрастного рейтинга (G, PG, PG-13, R, NC-17), интерактивный селектор тегов (меток) для категоризации контента, большую текстовую область для ввода основного содержания истории с placeholder «Начните писать...», кнопку «Изменить текст с помощью AI» для доступа к AI-ассистенту, а также кнопку «Создать историю» для публикации произведения. При нажатии на кнопку AI-редактирования пользователь перенаправляется на экран детальной настройки AI-улучшений. Пользователь может выбрать несколько тегов из предопределённого списка: «Фэнтези», «Научная фантастика», «Драма», «Романтика», «Детектив», «Киберпанк», «Приключения», «Мрачное», «Нуар». Выбранные теги визуально выделяются тёмным фоном. При сохранении истории StoryEditorViewModel отправляет POST-запрос на эндпоинт `/api/stories` с данными произведения. После успешного создания история публикуется в общем каталоге. Демонстрация внешнего вида экрана создания истории представлена на рисунке 14.

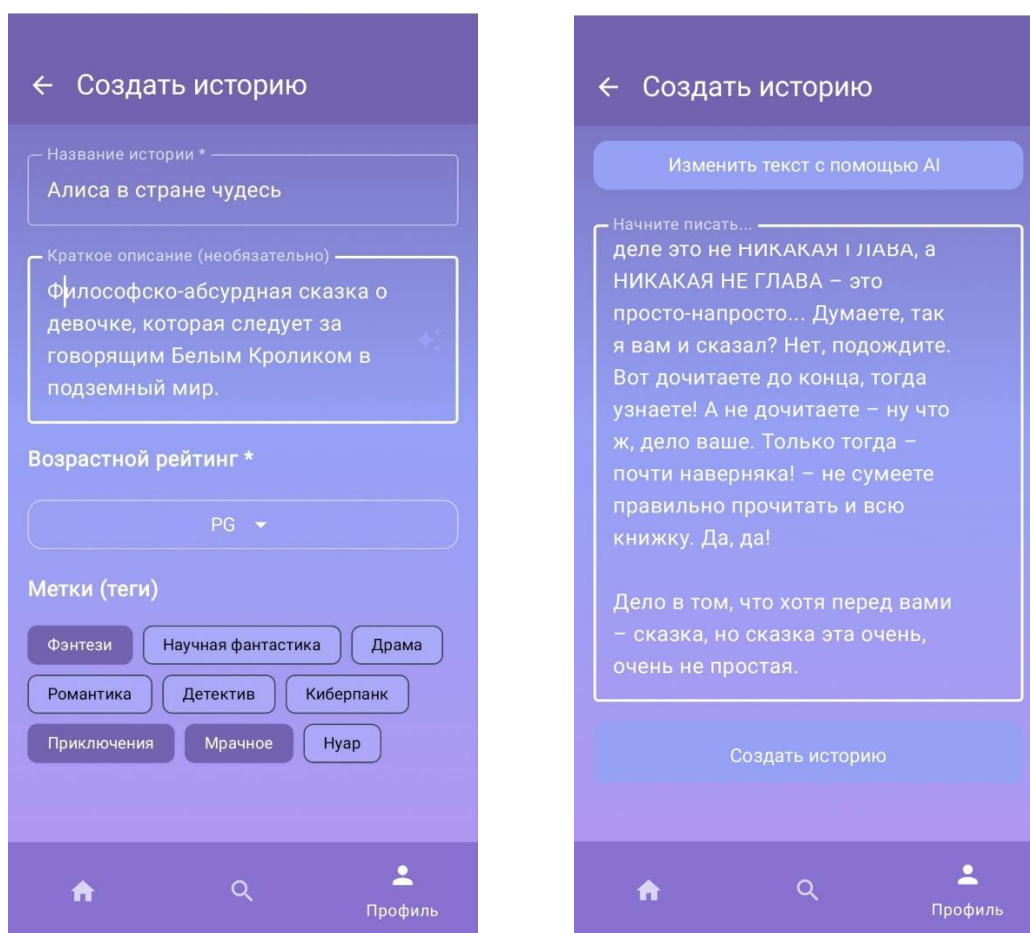


Рисунок 14 - Экран создания истории приложения Liboffan

3.2.3 Главный экран (Каталог историй)

Главный экран (HomeScreen) представляет собой каталог доступных историй и является центральным элементом навигации приложения. На экране отображается список карточек произведений, каждая из которых содержит: название истории, имя автора, теги жанров, а также раскрывающийся блок «Ответвления» с указанием количества веток. При нажатии на заголовок карточки происходит переход к детальному просмотру истории (BookDetailScreen). Блок ответвлений отображает все существующие ветки данной истории с указанием автора ветки, возрастного рейтинга и связанных тегов. Например, история «The Lost Forest» имеет три ответвления от разных авторов (Alice, Bob, Charlie), каждое из которых может иметь уникальный

набор тегов (fantasy + adventure, fantasy + dark и т.д.). Нижний навигационный бар обеспечивает быстрый переход между основными разделами приложения: «Главная», «Поиск», «Профиль». Внешний вид главного экрана представлен на рисунке 15.

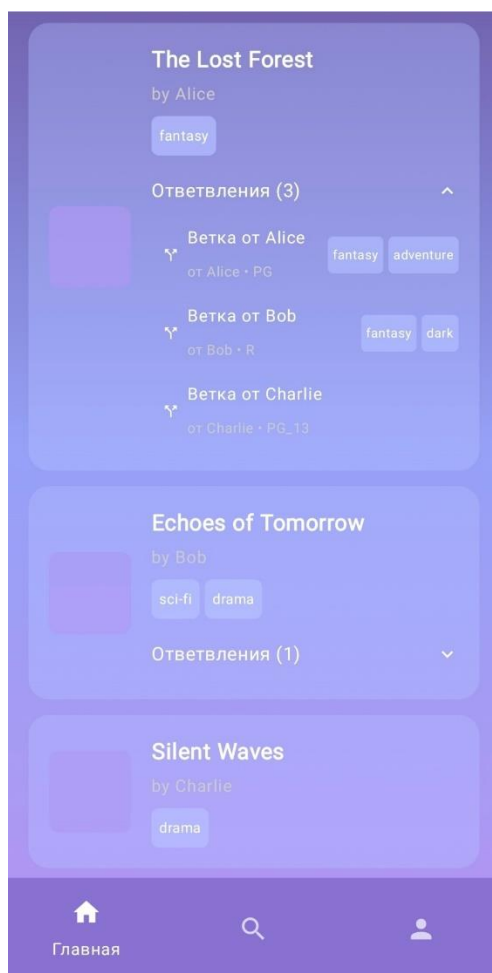


Рисунок 15 - Главный экран (каталог историй)

3.2.4 Экран поиска

Экран поиска (SearchScreen) предоставляет расширенные возможности фильтрации контента для удобного поиска историй по интересам. Интерфейс экрана включает: текстовое поле для поиска по названию произведения, выпадающий список для выбора фандома, селектор возрастного ограничения

(например, PG-13), а также интерактивную сетку тегов для фильтрации по жанрам. Пользователь может комбинировать несколько фильтров одновременно: например, выбрать тег «Научная фантастика» и ограничение PG-13. При изменении параметров поиска SearchViewModel отправляет запрос на эндпоинт `/api/stories` с соответствующими query-параметрами. Результаты поиска отображаются динамически в нижней части экрана с указанием количества найденных историй. Выбранные фильтры визуально выделяются тёмным фоном для удобства пользователя. Демонстрация внешнего вида экрана поиска представлена на рисунке 16.

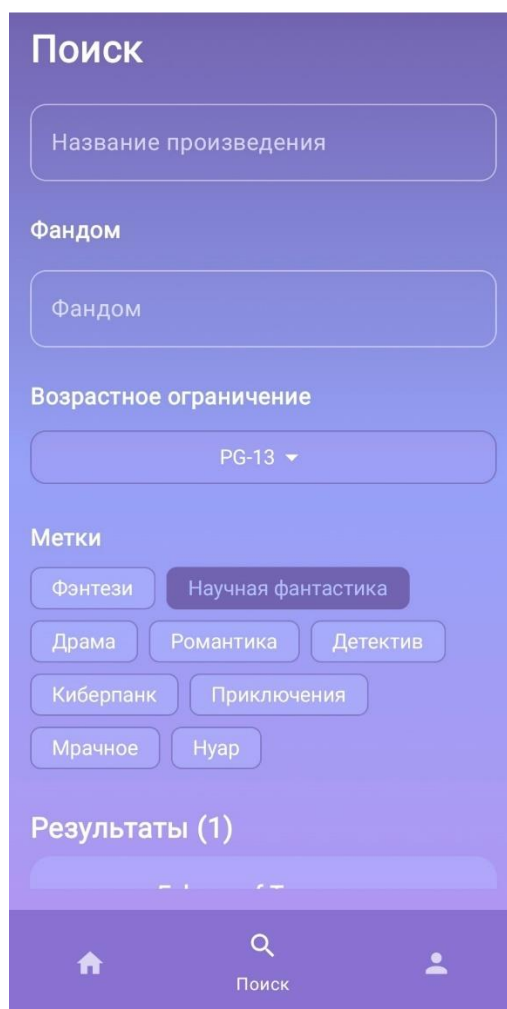


Рисунок 16 - Экран поиска

3.2.5 Экран профиля и библиотеки

Экран профиля (ProfileScreen) объединяет управление аккаунтом и персональной библиотекой пользователя. В верхней части экрана отображается аватар пользователя (с возможностью загрузки), имя аккаунта и кнопка меню для доступа к настройкам. Центральная часть экрана содержит вкладки библиотеки с горизонтальной навигацией: «В планах» (planned), «В процессе» (reading), «Прочитано» (completed), «Заброшено» (dropped). Каждая вкладка отображает список историй, добавленных пользователем с соответствующим статусом чтения. Карточка истории в библиотеке содержит: название, имя автора, краткое описание (синопсис), тег жанра, а также кнопку удаления из библиотеки. При нажатии на карточку происходит переход к чтению или просмотру деталей истории. ProfileViewModel управляет состоянием библиотеки, синхронизируя данные с эндпоинтом `/api/me/library`. Внешний вид экрана профиля представлен на рисунке 17.

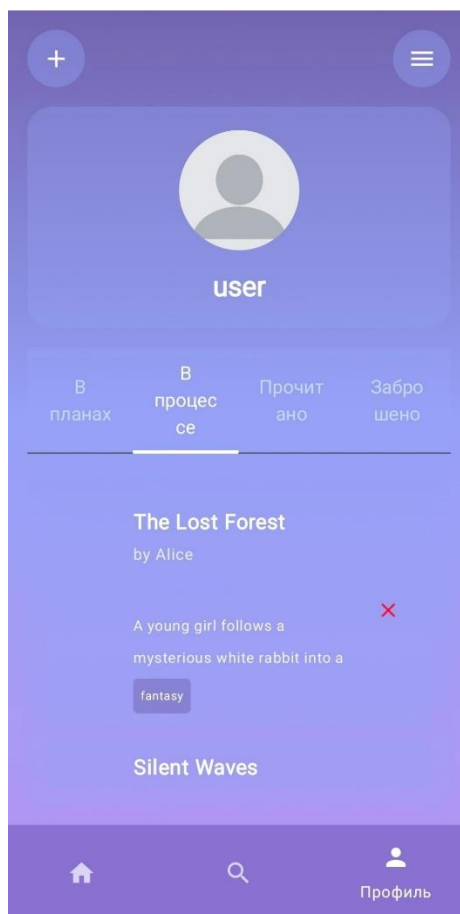


Рисунок 17 - Экран фотоальбома с фотографиями, сгруппированными по локациям

3.2.6 Экран AI-редактирования

Экран AI-редактирования (AIEditScreen) реализует функцию интеллектуальной обработки текста с использованием локальной языковой модели Ollama. Интерфейс экрана разделён на три функциональные зоны:

Верхняя секция отображает исходный текст, выбранный пользователем для редактирования (например: «Дело в том, что хотя перед вами – сказка, но сказка эта очень, очень не простая.»).

Средняя секция содержит поле ввода «Инструкция для AI:», куда пользователь вводит текстовый запрос на естественном языке (например: «сделай текст короче», «улучши стиль», «добавь драматизма»). Кнопка «Запросить

улучшение» отправляет POST-запрос на эндпоинт `/api/ai/edit` с параметрами: исходный текст и инструкция. AI-модель обрабатывает запрос и возвращает улучшенную версию текста.

Нижняя секция «Редактирование вручную:» предоставляет текстовую область с результатом AI-обработки, которую пользователь может дополнительно отредактировать перед сохранением. В примере показан сокращённый вариант текста: «Я обманул вас: это не Никакая Глава, а Никакая Не Глава. До-читаете – узнаете. Эта сказка – не простая.»

Кнопка «Сохранить изменения» в нижней части экрана фиксирует отредактированный текст и возвращает пользователя к основному редактору. `AIEditViewModel` координирует взаимодействие с сервером, обрабатывает состояния загрузки и ошибки сети. Такая реализация позволяет авторам быстро улучшать текст, не покидая приложения, при этом все данные обрабатываются локально на сервере, обеспечивая конфиденциальность творческого процесса. Демонстрация внешнего вида экрана AI-редактирования представлена на рисунке 18.

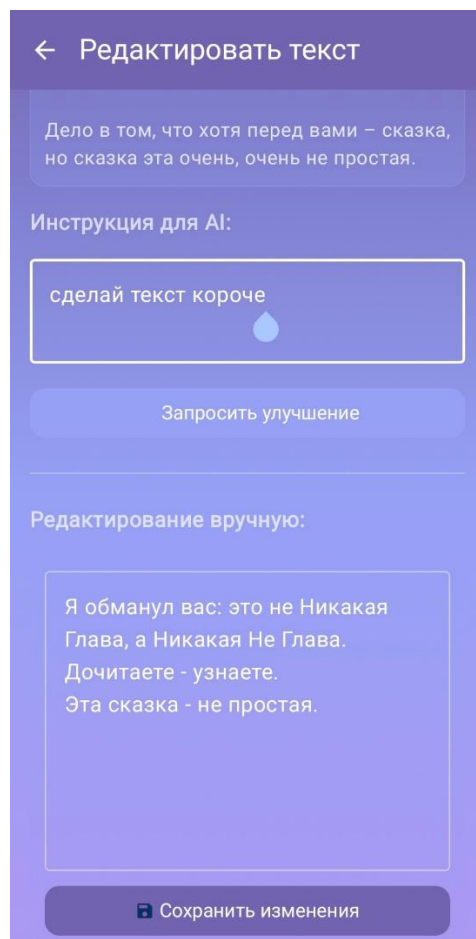


Рисунок 18 - Экран AI-редактирования текста

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было разработано мобильное приложение «Liboffan» — платформа для коллаборативного создания и чтения историй с системой ветвления сюжетов.

Были решены все поставленные задачи: проведён анализ рынка и аналогов, выбран и обоснован стек технологий, спроектирована архитектура клиент-серверного взаимодействия, реализованы ключевые функции — от авторизации до AI-редактирования текста.

Особенностью системы является реализация концепции версионирования историй, заимствованной из практики разработки ПО, но адаптированной для литературного творчества. Это позволяет авторам экспериментировать с сюжетом, а читателям — выбирать предпочтительное развитие событий, не теряя связи с оригиналом.

Практическая значимость работы заключается в создании готового к использованию продукта, который закрывает существующий пробел между пассивным чтением и активным соавторством. Приложение может быть использовано как самостоятельный продукт или как основа для образовательных проектов в области литературного мастерства.

Перспективы развития включают внедрение совместного редактирования в реальном времени, поддержку мультимедийного контента (иллюстрации, аудио), расширение AI-функционала (генерация идей, проверка грамматики) и адаптацию для веб-платформы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. What is Kotlin Multiplatform. – Текст: электронный // kotlinlang.org: [сайт]. – 2025. – URL: <https://kotlinlang.org/docs/multiplatform/kmp-overview.html> (дата обращения: 21.02.2025).
2. Jetpack Compose и Kotlin: как разрабатывать современные UI. – Текст. : электронный // Тпрогер : [сайт]. – 2025. – URL: <https://tproger.ru/articles/jetpack-compose-i-kotlin--kak-razrabatyvat-sovremennye-ui> (дата обращения 20.02.2026)
3. Паттерн MVVM. – Текст. : электронный // METANIT: [сайт]. – 2016. – URL: <https://metanit.com/sharp/wpf/22.1.php> (дата обращения 19.02.2026)
4. Как библиотека Retrofit 2 решает сетевые трудности перевода - METANIT. – Текст. : электронный // Яндекс практикум: [сайт]. – 2024. – URL: <https://practicum.yandex.ru/blog/retrofit-na-android/> (дата обращения 19.02.2026)
5. Фреймворк Spring: зачем он нужен, как устроен и как работает. – Текст. : электронный // skillbox: [сайт]. – 2022. – URL: <https://skillbox.ru/media/code/freymvork-spring-zachem-on-nuzhen-kak-ustroen-i-kak-rabotaet/> (дата обращения 20.02.2026)
6. Первые шаги в Spring Security с JWT. – Текст. : электронный // habr: [сайт]. – 2016. – URL: https://habr.com/ru/companies/spring_aio/articles/909448/ (дата обращения 25.02.2026)
7. MariaDB. – Текст. : электронный // workspace: [сайт]. – 2018. – URL: <https://workspace.ru/tools/database/mariadb/> (дата обращения 19.02.2026)
8. Руководство по Spring Data JPA. – Текст. : электронный // arenda server: [сайт]. – 2024. – URL: <https://arenda-server.cloud/blog/rukovodstvo-po-spring-data-jpa/> (дата обращения 19.02.2026)

9. Ollama: установка и настройка на ПК — как запустить Llama/другие модели локально. – Текст. : электронный // кодик : [сайт]. – 2026. – URL: <https://itcodik.com/article/ollama-ustanovka-nastroika-zapusk-llama-lokalno> (дата обращения 15.04.2026)
10. Android Developers. Guide to app architecture (MVVM). – Текст: электронный // developer.android.com: [сайт]. – 2024. – URL: <https://developer.android.com/topic/architecture> (дата обращения: 16.02.2025).
11. Android Developers. Jetpack Compose. – Текст: электронный // developer.android.com: [сайт]. – 2024. – URL: <https://developer.android.com/compose> (дата обращения: 15.01.2025).